

TermAld

ORCHESTRATION PLAYBOOK

How to run the project as a set of focused chat windows with this main thread as the integration hub and single source of truth.

You are the orchestrator. This window is the ops desk. Each focused window owns one slice of the code. Work flows out as kits and comes back as hand-offs that get folded into the master before going to GitHub.

Platform: 120 modules • 1,949 commands • backend (Python) / frontend (TypeScript) / native (Rust) / desktop-mobile (Tauri) / CI-CD
Document version 1.0

1 The Operating Loop

1. ISSUE	This window hands you a kit for a window: a PDF brief, the complete file system + source, and the master index.
2. ASSIGN	You open a new window, paste the start prompt from the brief, and attach that window's files. You set the day's task.
3. WORK	The focused window does only its slice. At the end it produces a hand-back : a PDF of what changed, the complete new file system + code, and an updated index entry.
4. RETURN	You bring that hand-back to this window.
5. INTEGRATE	This window merges it: updates the master index + ARCHITECTURE, reconciles any cross-window effects, and regenerates the affected source.
6. PUSH	This window hands you the updated source + index. You commit and push to GitHub.

The middleman is you. Nothing syncs by itself — that is the point. The master index is what keeps every window honest.

2 Roles & Daily Cadence

Orchestrator (you)

Pick the window of the day. Carry kits out and hand-backs in. Commit to GitHub. Decide priorities.

Ops desk (this window)

Keeps the master index + full source. Generates per-window kits. Integrates hand-backs, resolves cross-window conflicts, bumps the version, returns the updated source.

Focused window (database, ai, docker, ...)

Works one slice only. Never edits another window's files. Ends every session by producing a hand-back in the standard format below.

3 The Hand-Back Contract

Every focused window returns the SAME three things so integration is mechanical, not guesswork:

- **HANDOFF.pdf (or .md)** — what changed and why
- **files/** — the complete new file system for that window's slice
- **INDEX.md** — the updated master-index entry for that window

HANDOFF template (fill this out at the end of every window session):

```
Window: <name>
Date / session: <date>
Version in: <e.g. 2.3.1> Version out: <2.3.2>

CHANGED:
- file path - one line on what changed

ADDED / REMOVED:
- new or deleted files

DECISIONS:
- any architectural choice the other windows must know

CROSS-WINDOW IMPACT:
- e.g. 'added column X -> backend schemas.py must add field'

NEW TODOs:
- follow-ups for this or other windows

TESTS: pass / fail / added
```

The CROSS-WINDOW IMPACT line is the most important one — it is how this desk knows what else to touch when integrating.

4 Versioning & GitHub Flow

Use simple semantic-ish versions on the platform: MAJOR.MINOR.PATCH. Each integrated hand-back bumps PATCH (or MINOR for a new feature). The version lives at the top of MASTER_INDEX.md so every window can see it.

Suggested branch-per-window GitHub flow:

```
# one branch per window keeps history clean git checkout -b window/database # ...paste the
integrated files from the ops desk... git add -A git commit -m "database: " git push -u origin
window/database # open a PR into main; merge once it builds (CI runs on push)
```

Because CI (ci.yml) runs Python + TypeScript + Rust on every push, a broken integration is caught before it reaches main.

5 The Windows

One synopsis per window. When you open it, paste its start prompt and attach its files. Numbers are a suggested build order (dependencies first).

0 · Main / Architecture

Role: Integration hub (this window)

Sole focus: Holds the master index + full source; issues kits; integrates hand-backs; owns versioning.

Owns: MASTER_INDEX.md, ARCHITECTURE.md

Depends on: —

1 · Database & Data Structures

Role: Data engineer

Sole focus: Schema, migrations, ORM models, data shapes. The foundation everything else reads.

Owns: backend/models.py, backend/database.py, backend/migrations/, modules/_shared/db.py

Depends on: Backend, AI, Knowledge (they read the schema)

2 · Backend Core & API

Role: Backend engineer

Sole focus: FastAPI app, routes, WebSocket, the engine that loads 120 modules, dispatch, rate limit.

Owns: backend/main.py, backend/engine.py, backend/schemas.py

Depends on: Database, Auth, AI

3 · Auth & Security

Role: Security engineer

Sole focus: JWT + passwords, the deployment policy (safe/system/dangerous), module gating.

Owns: backend/auth.py, backend/policy.py; modules: security, sec, privesc, perms, sandbox

Depends on: Backend, Database

4 · AI & The Brain

Role: AI engineer

Sole focus: Providers (12), token streaming, reasoning modules. The model layer.

Owens: backend/ai_stream.py, backend/providers_extra.py, termaid/providers/; modules: brain, cognition, cortex, smart, agent, chain

Depends on: Backend, Secrets/Config

5 · Knowledge & Learning

Role: Knowledge / ML engineer

Sole focus: Memory, the knowledge base, learning & deep-learning features.

Owens: modules: memory, learn, learner, lessons, notes, catalog

Depends on: AI, Database

6 · Networking & Scanning

Role: Network engineer

Sole focus: Network analysis + the fast Rust scanner. Local-mode only.

Owens: modules: netscan, nettools, netdeep; native/ scanner

Depends on: Native/Rust, Security

7 · Native / Rust Performance

Role: Systems engineer

Sole focus: The Rust hot-path crate (scan + walk) and Tauri native commands.

Owens: native/, desktop-mobile/src-tauri/src/lib.rs

Depends on: Backend (sidecar), Frontend (bridge)

8 · Frontend / UI

Role: Frontend engineer

Sole focus: TypeScript terminal UI, typed API + WS clients, the native bridge.

Owens: frontend/src/ (main, api, ws, terminal, native, types)

Depends on: Backend (API contract), Native

9 · Desktop & Mobile (Tauri)

Role: App engineer

Sole focus: Compile the UI into native desktop + mobile; bundle the local backend sidecar.

Owns: desktop-mobile/, scripts/build_sidecar.*, backend/sidecar.py, backend/runtime.py

Depends on: Frontend, Native, Backend

10 · Docker / Deploy / CI-CD

Role: DevOps

Sole focus: Containers and the GitHub Actions pipelines (test + release + bundled).

Owns: Dockerfile, docker-compose.yml, .github/workflows/, scripts/

Depends on: All (it builds them)

11 · Secrets & Config

Role: Config

Sole focus: OS keychain for API keys, the .env loader, settings.

Owns: backend/secrets.py, backend/settings.py, .env.example

Depends on: AI, Backend

12 · Modules System & Engine

Role: Platform internals

Sole focus: How the 120 modules discover/load; the command registry; classify the 7 uncategorised modules.

Owns: termaid/extensions/, modules/_shared/; modules: api, auth, dev, health, pdf, ratelimit, repl

Depends on: Backend, Database

13 · Testing & QA

Role: QA engineer

Sole focus: Grow the test suites across Python + Rust; keep CI green.

Owns: backend/tests/, native/tests/

Depends on: All

6 Daily Checklist

Start of day

- Pick the window of the day from the roster.
- Open it; paste its start prompt; attach its files + the current MASTER_INDEX.
- State the single task for the session.

End of session (in the focused window)

- Produce HANDOFF + complete files/ + updated INDEX.md.
- Confirm tests pass (or note what's pending).

Back at the ops desk (this window)

- Hand me the three hand-back pieces.
- I integrate, bump the version, flag cross-window effects, return updated source + index.
- You commit on a window/<name> branch and push; let CI verify.

Keep MASTER_INDEX.md and ARCHITECTURE.md in the Project knowledge so every window starts oriented. This playbook is the rulebook; the master index is the live state.